Neifit

Sabores tradicionales.
Una experiencia digital cercana.



Tienda virtual

Postres · Snacks · Bebidas

Presenta

Maribel Marín · 10 de noviembre de 2026

Una venta necesita más que un mensaje.

El problema que busca resolver Neifit

01

Información dispersa

Solicitudes, datos del cliente y productos en registros separados.

02

Poco control

Dificultad para conocer cantidades, disponibilidad y solicitudes pendientes.

03

Seguimiento limitado

El cliente necesita saber qué ocurrió con su pedido y su entrega.

Centralizar la información para atender, organizar y dar seguimiento.

De conocer la marca a recibir el pedido.

Una plataforma web con acceso según la función de cada persona

01 · 01

Explorar

Landing institucional y muestra de productos.

02 · 02

02 · Ingresar

Registro e inicio de sesión con Supabase Auth.

03 · 03

03 · Solicitar

Carrito, pedido o cotización según la necesidad.

04 · 04

04 · Gestionar

Administración y seguimiento de la entrega.

React + Vite → FastAPI → Supabase / PostgreSQL: arquitectura objetivo.

Una plataforma. Responsabilidades distintas.

¿Quiénes la utilizarían?

01

Cliente

Consulta productos, solicita cotizaciones, crea pedidos y consulta los propios.

02

Administrador

Gestiona productos, solicitudes, usuarios, roles y permisos.

03

Mensajero

Consulta entregas asignadas y registra avances o novedades.

Los roles personalizados amplían las funciones mediante permisos autorizados.

¿Qué necesita conocer el sistema?

La información se organiza en cuatro grupos

Personas y acceso *01*

Nombre, correo, teléfono, rol y permisos.

Catálogo *02*

Categorías, productos, precios, ingredientes, alérgenos y stock.

Operación *03*

Carritos, pedidos, cotizaciones, detalles y direcciones.

Control *04*

Seguimiento, transacciones de pago y auditoría de accesos.

Las contraseñas se administran en Supabase Auth, no en profiles.

La base del modelo.

Entidades principales del catálogo y de los usuarios

Entidad	Atributos principales	Función
profiles	id, full_name, email, phone, role	Identidad operativa del usuario
categories	id, name, slug, is_active	Agrupar productos
products	id, category_id, name, price, stock, image_url	Describir la oferta y su disponibilidad
carts / cart_items	user_id, status / product_id, quantity, unit_price	Organizar la intención de compra

Cada registro tiene identidad propia y referencias a las entidades relacionadas.

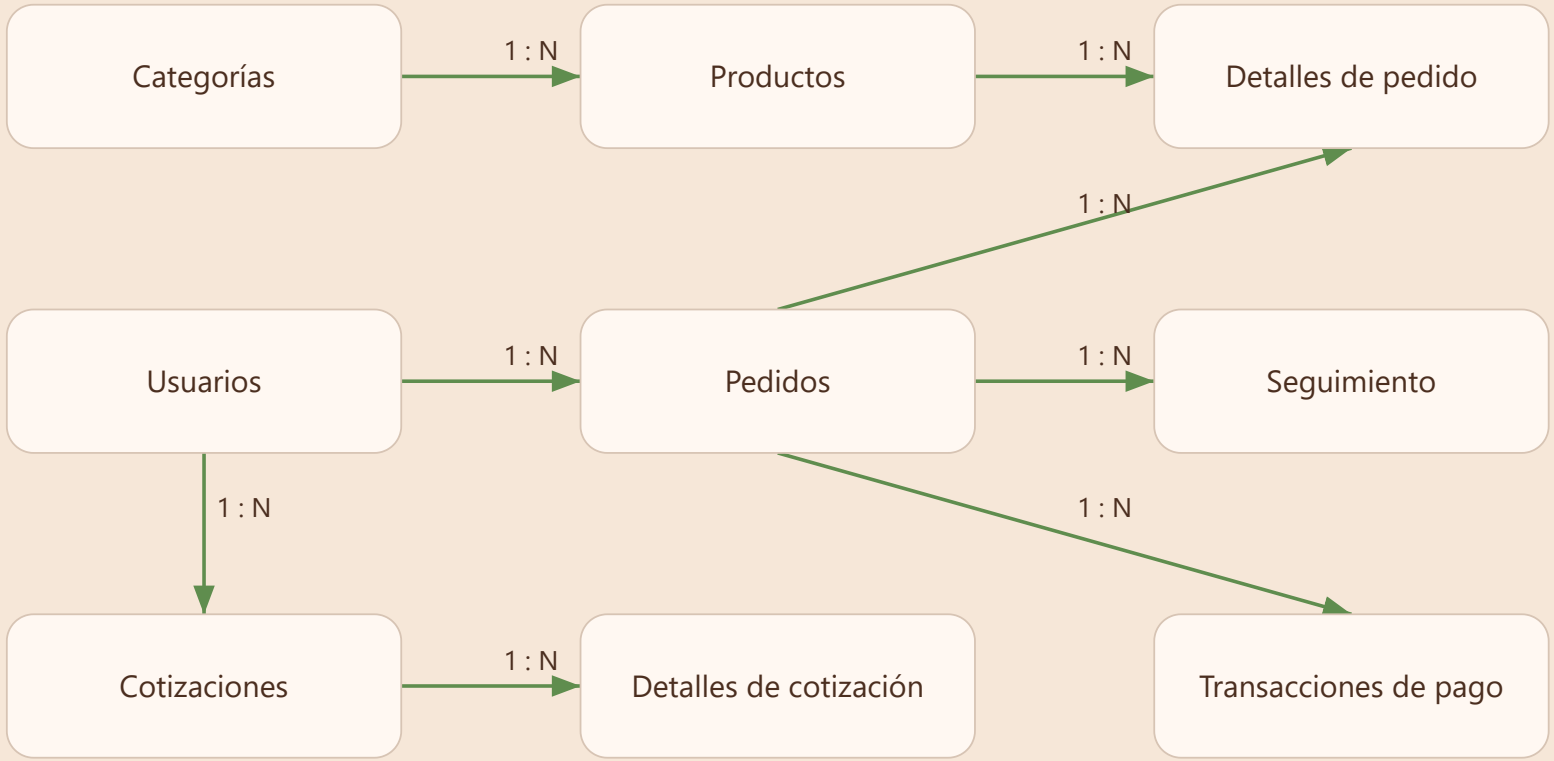
Cada solicitud conserva su detalle.

Entidades transaccionales del modelo diseñado

Entidad	Atributos principales	Función
orders	id, user_id, order_number, address, order_status, total	Registrar una compra
order_items	order_id, product_id, product_name, quantity, unit_price	Conservar los productos y precios del pedido
quotes / quote_items	quote_number, user_id, status / quantity, reference_price	Registrar una solicitud de cotización

Las relaciones dan coherencia al negocio.

Esquema conceptual simplificado · 1 = uno · N = muchos



El acceso depende de lo autorizado.

Permisos con nombres concretos para cada responsabilidad

Módulo o acción	Permiso	Alcance
Catálogo	catalog.read	Consultar productos
Pedidos propios	orders.read_own	Solo registros del cliente
Entregas asignadas	delivery.read_assigned	Solo pedidos asignados
Productos e inventario	products.manage	Gestión autorizada
Usuarios / Roles	users.manage / access.manage	Reservados al administrador

Reglas que protegen la información.

Integridad de datos y autorización

Datos válidos 01

Precios y cantidades positivos. Stock no negativo. Estados permitidos.

Referencias consistentes 02

Claves primarias y foráneas. Números de pedido y cotización únicos.

Compras coherentes 03

Un carrito activo por usuario. Un producto por detalle de cada documento.

Acceso controlado 04

RLS en tablas privadas. Sin cambio del rol propio. Entregas por asignación.

Un modelo relacional para crecer con orden.

¿Por qué se diseñó de esta manera?

01

Separar responsabilid

Usuarios, productos, documentos y detalles tienen funciones distintas.

02

Conservar el contexto

El precio del pedido no cambia cuando cambia el catálogo.

03

Mantener trazabilidad

Cada avance de entrega y cambio de acceso puede quedar registrado.

Menos duplicación innecesaria, mejores relaciones y consultas más claras.

De los registros a decisiones útiles.

Ejemplos sobre las tablas del modelo, cuando tengan datos persistidos

Inventario bajo

¿Qué productos necesitan reposición?

Solicitudes pendientes

¿Qué cotizaciones faltan por atender?

Operación de entregas

¿Qué pedidos siguen sin entregar?

SQL / INVENTARIO

```
SELECT name, stock
FROM public.products
WHERE is_active = true
AND stock <= 5
ORDER BY stock ASC;
```

Prototipo funcional, con una ruta de evolución.

Diferenciar lo implementado de lo pendiente

01

Implementado en el código

Interfaz y módulos comerciales. Autenticación con Supabase. Controles de acceso en la API. Gestión de roles preparada con SQL y configuración.

02

Pendiente para producción

Persistir pedidos y cotizaciones en Supabase. Validar stock y totales con transacciones. Integrar y verificar pagos reales. Completar pruebas de operación.

Hoy los pedidos y las cotizaciones siguen en memoria y

Organizar la venta. Acompañar cada entrega.

Neifit conecta clientes, administración y mensajería en una misma experiencia.

1 · Demostrar

Recorrer catálogo, cotización y panel administrativo.

2 · Consolidar

Conectar los documentos comerciales a PostgreSQL.

3 · Validar

Comprobar permisos, inventario y pagos de extremo a extremo.

Gracias · Preguntas